Internet Technology Assessment

Individual and Group Work Assessment

Students of the internet technology module must successfully pass the following two types of assessments:

1. The individual **written exam** takes place during the official exam period and must be done using the FHNW School of Business iPad exam infrastructure and system. The iPad exam, which is closed book, is graded with a mark between 1 and 6 and counts 50% to the overall module grade.
2. The group **project** is graded with a mark between 1 and 6 and counts 50% to the overall module grade. Assessed will be the demonstrator, a video, and an online documentation. This group project leads to a group grade. If a person's participation is significantly different from the group's contribution, the evaluators may provide a separate evaluation.

In the following, this document contains further information about the group project and its conditions.

1 Background of the Group Project

The underlying idea of this group project is that students can practice and transfer the knowledge gained from the lectures into a working Web application. Coaching sessions will be provided as part of the lecture schedule.

2 Assessment of the Group Project

The team performance will be assessed based on group work. A group work project team (2-4 students) must engineer a web application based on an own project idea reflecting the following user stories, requirements and constraints.

2.1 Minimal generic user stories

In the following, there are minimal generic user stories listed, which may be adapted or extended in a way that it is fitting to use case or application scenario.

1. As an [admin], I want to have a Web app so that I can use it on different mobile devices and on desktop computers.
2. As an [admin], I want to see a consistent visual appearance so that I can navigate easily, and it looks consistent.
3. As an [admin], I want to use list views so that I can explore and read my business data.
4. As an [admin], I want to use edit and create views so that I can maintain my business data.
5. As an [admin], I want to log-in so that I can authenticate myself.
6. As a [user], I want to use list views so that I can access public pages.
7. (Optional) As a [user], I want to authenticate myself so that I can read my personal and confidential data.

2.2 Strict Generic Requirements

The following high-level requirements must be fulfilled in the group work:

- **Architecture**: The Web app must reflect at least three different layers on at least two tiers (backend and frontend).
- **Views**: The generic user stories must be realized with **at least four different views**.
- **User data**: The database schema must consist of **at least four entities**.
- **Business logic**: The service layer should implement **at least one business rule** which corresponds to an operation or a constraint of an enterprise.

2.3 Constraints

Each team must consider the following constraints:

- **Web-Design**: The web application must be carefully designed concerning consistent identity and design, adequate user experience, qualitative graphics, responsive layout and well-chosen typography. As a recommendation, use a low code tool such as Budibase or any other web app/form designer that can consume REST APIs.
- **Design principles**: It is strongly recommended to use the power of OOP, design and architectural patterns, API design principles, routing functionalities, DRY and CRUD paradigm.
- **Frontend Web Technology**: It is recommended to use a low-code app design tool like Budibase to implement the front-end functionality. However, a more code-based approach (full-code) may be used if the required functionality cannot be achieved using a low-code tool. In such cases, the choice must be justified with a technical explanation and discussed with the lecturer for confirmation.
- **Backend Web Technology**: The backend must be implemented using an enterprise-grade framework and programming language (at least Spring-Boot 3.0 and Java 17). However, other backend technologies may be used if there is a technological justification, and the choice is discussed and approved by the lecturer.
- **Database Technology**: The reference project uses H2, a lightweight in-memory relational database, making it recommended for demonstration purposes. However, any other relational database (e.g., MySQL) or Graph DB technology may be used if there is a justified technical reason, such as scalability or performance needs.
- **Source Code and Artefacts**: The source code and further required artefacts such as images, SQL scripts, libraries, or dependency information must be under version control by using GitHub. Each team must create and share their GitHub repository early in the project to allow tracking of progress and version history. The repository should be maintained actively throughout the development process.
- **OpenAPI Documentation**: The API end points in the web service must be documented and made available using OpenAPI 3.0 (Swagger) documentation.
- **Documentation/report**: The web engineering process, including requirements analysis, design decisions, implementation, and installation, must be clearly documented. This documentation is required to be included in the README file on GitHub, detailing the group composition, link to the video presentation, link to the deployed web application (if applicable), and link to the OpenAPI documentation. The remaining sections of the README must cover the topics outlined in Milestones (Section 2.5), ensuring a structured and comprehensive project overview.
- **Presentation video**: All teams must present their work in a video of 10 minutes max. The video may start with the use case, and then it continues with the design, followed by a demonstration of the running Web app. Take the audience on a (customer) journey. Teams may pick out one or two fancy or impressive things they are proud of and highlight them. It is not intended to show the source code in the video.
- **Teamwork**: All team members must be involved somehow; in the sense of teamwork, the teams should assign various tasks to all team members.

2.4 Deliverables

The following deliverables are mandatory:

- Source code and artefacts (link and export for front-end application e.g. from Budibase, link to GitHub repository for the web service; invite lecturers if using a private repository).
- Documentation/report (link to GitHub Markdown file(s))
- Deployment on a web server is not expected. But published web application and web services configured and running on GitHub Codespaces are required.
- Presentation video (link to a video hosted on Microsoft Stream, SWITCHtube or YouTube; restricted/unlisted access possible and recommended)

Detailed instructions on the submission of the deliverables are indicated on Moodle LMS.

2.5 Milestones

1. Analysis: Scenario ideation, use case analysis and user story writing.
2. Domain Design: Definition of domain model.
3. Frontend implementation: Design, prototyping and realization of frontend functionality.
4. Business Logic and API design: Definition of business logic and API.
5. Data and API implementation: Implementation of data access and business logic layers and API.
6. Security: Implementation of API-level security (Basic Auth).
7. Demonstrator: Integration of frontend and backend to realize an end-to-end application consuming REST APIs from the web service.

2.6 Deadlines and submission

The deadline for submitting the previously mentioned links to the deliverables is **indicated in the Moodle LMS**. The submission can be made by providing the links in the corresponding Moodle course prior to the deadline by one group member. Again, t*he names of the group work members and their contribution must be disclosed in the documentation.*

Appendix: Evaluation Sheet

Internet Technology Group Work Assessment

Student group:	
Project:	
Examiner:	

Criteria	Things to look for	Weight	Grades (1-6)	Weighted Points	Comments
Scenario and User Experience (UX)	Project context, domain and scenario is defined appropriately. Key use cases are defined, realistic and reflect the generic user stories. Scenario is innovative and user experience is addressed.	10%			
Requirements Analysis	Relevant aspects named and appropriately weighted. User stories cover all relevant aspects of the use case.	10%			
Completeness, Design and Architecture	Functions implemented according to task. Adequate design (inclusion of OOP, design and architectural patterns, API design principles, DRY and CRUD paradigm). Constraints reflected.	15%			
Creativity and Web-Design	Effort invested in the project, design of the user interface, overall creativity impression.	15%			
Artefacts and Implementation	Artefacts are of good quality. Artefacts are available and self-explanatory. Cloud-based instantiation is running and available. Instantiation, solution and results are reproducible.	20%			
Digital Data	Appropriate data model, structured data and/or semantics.	10%			
Project Management and Documentation	Time planning and cooperation. Communication and information. Logical structure and clarity of documentation. Graphical appearance of documentation.	5%			
Presentation/Video	Good quality of presentation video. Clarity and good style of presentation. Students are convincing and competent.	15%			
Total Points (max. 100)		**100%**			

Concluding remarks

Final Grade	